LLM-Based User Simulation for Low-Knowledge Shilling Attacks on Recommender Systems

Shengkang Gu*
Jiahao Liu*
Fudan University
Shanghai, China
gusk24@m.fudan.edu.cn
jiahaoliu23@m.fudan.edu.cn

Dongsheng Li
Microsoft Research Asia
Shanghai, China
dongsli@microsoft.com

Guangping Zhang
Fudan University
Shanghai, China
gpzhang20@fudan.edu.cn

Mingzhe Han
Fudan University
Shanghai, China
mzhan22@m.fudan.edu.cn

Hansu Gu
Independent
Seattle, United States
hansug@acm.org

Peng Zhang
Fudan University
Shanghai, China
zhangpeng_@fudan.edu.cn

Ning Gu
Fudan University
Shanghai, China
ninggu@fudan.edu.cn

Li Shang
Fudan University
Shanghai, China
lishang@fudan.edu.cn

Tun Lu
Fudan University
Shanghai, China
lutun@fudan.edu.cn

arXiv:2505.13528v1 [cs.IR] 18 May 2025

Abstract

Recommender systems (RS) are increasingly vulnerable to shilling attacks, where adversaries inject fake user profiles to manipulate system outputs. Traditional attack strategies often rely on simplistic heuristics, require access to internal RS data, and overlook the manipulation potential of textual reviews. In this work, we introduce Agent4SR, a novel framework that leverages Large Language Model (LLM)-based agents to perform low-knowledge, high-impact shilling attacks through both rating and review generation. Agent4SR simulates realistic user behavior by orchestrating adversarial interactions—selecting items, assigning ratings, and crafting reviews—while maintaining behavioral plausibility. Our design includes targeted profile construction, hybrid memory retrieval, and a review attack strategy that propagates target item features across unrelated reviews to amplify manipulation. Extensive experiments on multiple datasets and RS architectures demonstrate that Agent4SR outperforms existing low-knowledge baselines in both effectiveness and stealth. Our findings reveal a new class of emergent threats posed by LLM-driven agents, underscoring the urgent need for enhanced defenses in modern recommender systems.

CCS Concepts

• **Information systems → Recommender systems**.

Keywords

Recommender Systems, Shilling Attacks, Large Language Models

ACM Reference Format:
Shengkang Gu, Jiahao Liu, Dongsheng Li, Guangping Zhang, Mingzhe Han, Hansu Gu, Peng Zhang, Ning Gu, Li Shang, and Tun Lu. 2025. LLM-Based User Simulation for Low-Knowledge Shilling Attacks on Recommender Systems. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 11 pages. https://doi.org/XXXXXXX.XXXXXXX

*Equal contribution.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

With the rapid growth of e-commerce, **R**ecommender **S**ystems (RS) have become essential for personalized content delivery [26–29, 33, 38]. For users, RS leverage interaction data, particularly ratings and reviews, to predict their preferences for specific items [13, 30–32]. For content providers (*e.g.*, online retailers), RS help match their items with interested users, thereby enhancing item exposure and driving revenue growth [2, 18]. As a result, malicious actors may attempt to manipulate RS to maximize their own interests. A common tactic is ***shilling attack***, where malicious user profiles with fabricated interaction data are injected into the system's training data to influence the outcome of RS [19, 23, 24, 44].

The essence of shilling attacks lies in exploiting distributional differences between fake and genuine users to influence RS performance. **Existing shilling attack methods exhibit several limitations:** (1) Conventional shilling attacks assign extreme ratings to target items: maximal for promotion (push attack) or minimal for demotion (nuke attack). For non-target items, attackers typically select them randomly and assign ratings using strategies like random values or item averages to mimic genuine user behavior [3, 16, 19, 34, 35, 51]. These rule-based profiles often lack diversity, making them easily detected as anomalies. (2) Some methods enhance camouflage by integrating additional external knowledge. For example, methods based on **G**enerative **A**dversarial **N**etworks

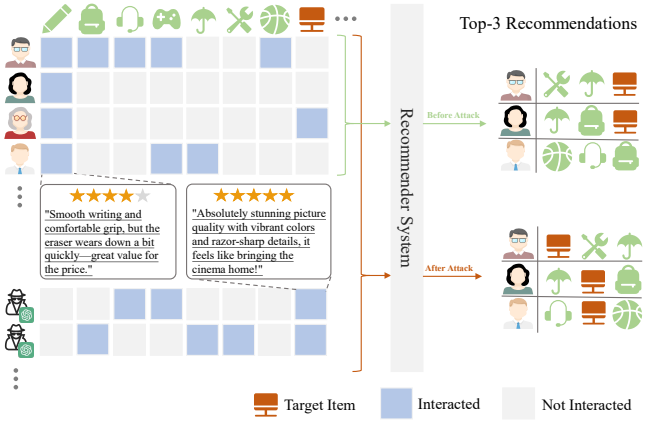

Figure 1: Overview of Agent4SR. As shown, injecting fake user profiles generated by LLM-based user agents manipulates recommendations and substantially boosts the target item's ranking.

(GAN) [12] generate fake user profiles by sampling patterns from genuine user interaction data [6, 7, 23, 24, 39, 43, 46, 52]. These models produce stealthier fake profiles and train a discriminator on both real and synthetic interaction data. Their main limitation lies in their heavy reliance on internal RS data (*e.g.*, genuine users' interactions), which is typically inaccessible in real-world attack scenarios. (3) Existing shilling attacks primarily manipulate rating data while often neglecting user reviews, which can carry greater risk due to their nuanced and semantically rich content [5, 54]. With advances in text mining and the increasing availability of user reviews, leveraging textual information in RS has become both common and effective [11]. Therefore, it is essential to investigate review-based attack strategies that complement rating-based approaches and to assess their potential impact on RS.

In recent years, **L**arge **L**anguage **M**odels (LLMs) have demonstrated strong capabilities in human-level reasoning, decision-making, and natural language generation [21]. This progress has motivated the development of LLM-based agents that simulate user behaviors using only minimal system-specific data. Consequently, several studies have proposed LLM-based agents to simulate users in RS [25, 47, 55, 57]. These agents serve multiple purposes, including bridging the gap between offline evaluation and online performance and modeling complex user behavior [15, 55]. However, the openness of RS exposes them to **malicious user agents**, either externally injected or derived from internally compromised LLM-based agents, both of which can severely undermine system integrity. Synthetic agents embedded with targeted malicious content can induce biased behaviors, including preferential promotion of specific vendors or suppression of certain product categories. By participating in activities such as rating and reviewing, these agents can manipulate recommendation outcomes in ways that serve the interests of their orchestrators.

In this paper, we propose **Agent4SR**, a novel low-knowledge shilling attack framework that employs LLM-based agents to manipulate RS. We inject a set of fake user agents into RS, which engage in adversarial yet human-like interactions by selecting items, rating them, and posting reviews to generate synthetic data that blends with real user interactions. This manipulation distorts the recommendation outcomes. For instance, in a promotion attack, the target item receives increased exposure, as illustrated in Figure 1.

Specifically, we designed tailored attack strategies for each component of the fake user agent. We developed a method for the profile module that infers and validates agent personality traits based on the target item and incorporates mechanisms to enhance diversity. In parallel, the memory module employs retrieval strategies that consider both relevance and recency. In the action module, beyond generating ratings, the agent is also capable of producing reviews, for which we designed a target feature propagation strategy. We simulated fake user interactions in the RS by structuring the information flow from the system into the agent.

We validated the effectiveness of Agent4SR across different types of RS, including those using ratings alone and those augmented with reviews. Agent4SR consistently outperformed existing low-knowledge attacks, highlighting its strong attack capability. The fake user agent exhibited strong behavioral resemblance to genuine users in our experiments, making it difficult to detect even with advanced detection algorithms. We conducted ablation experiments to evaluate the contribution of each component in Agent4SR. We also performed additional studies to investigate how fake users of varying scales, targeting items with different popularity levels, affect users across different activity tiers.

To the best of our knowledge, Agent4SR is the first to leverage LLM-based agents to simulate users for attacking RS, and we demonstrate that the resulting attacks exhibit both substantial harm and stealth. By leveraging the **human-like reasoning**, **rich world knowledge**, and **natural language generation capabilities** of LLMs, Agent4SR overcomes key limitations of existing approaches. Agent4SR introduces a paradigm shift in shilling attacks, moving from rule-based heuristics to cognitively simulated agent behavior. As the use of LLM-based agents becomes more widespread, our objective is to highlight emerging security risks posed by such agents acting like genuine users.

2 Related Work

2.1 Shilling Attacks against RS

Numerous methods have been proposed for black-box shilling attacks against RS. Early studies explored strategies that utilize global statistical information to generate fake user profiles for push attacks (*e.g.*, random, bandwagon, segmented) and nuke attacks (*e.g.*, reverse bandwagon, love/hate) [3, 19, 34, 35, 44]. To improve attack stealth, heuristic techniques such as noise injection, user shifting, target shifting, and popularity-based averaging have been proposed to obscure aggressive rating patterns and evade detection [16, 51]. These methods represent simple low-knowledge black-box attacks that rely on predefined rules and **fail to capture the behavioral complexity of genuine users in personalized contexts** which limits their stealthiness.

Several studies assume that attackers have access to more knowledge, which can be used to diversify malicious user data and enhance its stealth against detection. For example, some attacks leverage **detailed interaction features** such as item rating counts and user or item in-degree centrality to identify and target influential

nodes in the system [39, 52]. Other work considers scenarios where **partial knowledge of the RS structure is available** and uses surrogate models to predict attack outcomes and guide item selection [46]. Reinforcement learning has also been used to train an agent that inject fake interactions and optimize attack strategies based on **periodic feedback from the RS**, although such feedback is rarely available in practice [5, 43]. The emergence of GANs has further advanced this line of work by enabling the automatic generation of fake user profiles. One approach applies DCGAN to generate initial profiles, trains a discriminator **to distinguish real from fake ratings**, and refines the profiles using zero-order optimization [6, 7, 36]. Another line of work improves GAN-based attacks by sampling genuine user templates from **the rating matrix** and using a generator to produce fake ratings, with a discriminator optimizing a global objective that integrates multiple loss functions [23, 24]. R-Trojan [54] generates fake profiles by combining user ratings with fine-tuned review generation. However, its review generation **lacks explicit attack objectives** and still relies on internal data such as **the rating matrix** for training, limiting its applicability.

2.2 LLM-Based User Agents in RS

LLM-based agents in RS can be broadly categorized into two types. The first leverages LLMs to generate or enhance recommendations, referred to as recommendation agents [15, 40, 49, 50, 56, 58, 61]. The second focuses on simulating user behavior through LLMs, known as user agents. While some studies model user dialogues in conversational recommendation settings [10, 17, 48, 62, 63], our work emphasizes modeling user actions such as rating and reviewing to reflect realistic interaction behavior in RS.

RecAgent [47] and Agent4Rec [55] employ LLM-based agents that integrate **profile**, **memory**, and **action** modules to simulate user decision-making processes in RS. RAH [42] introduces LLM-based multi-agents that serve both as recommendation generators and as simulators of user behavior, bridging users and RS. FLOW [4] facilitates collaboration between recommendation and user agents via a feedback loop. Zhang et al. [60] integrate explicit preferences, LLM-based sentiment analysis, human-in-the-loop modeling, and statistical frameworks to simulate user interactions more robustly. AgentCF [57] introduces a co-learning framework that models users and items as agents and optimizes them jointly. AgentCF++ [25] extends this framework to support cross-domain transfer and incorporate popularity-aware modeling. Together, these works highlight the potential of LLMs for simulating human-like user behavior in RS.

3 Preminilaries

In this section, we formally define the shilling attack problem and introduce key related concepts.

3.1 Definition of Shilling Attack

Shilling attacks involve injecting fake user profiles into RS to manipulate recommendation outcomes.

Let the input to the RS be a user-item interaction dataset $\mathcal{D} = \{D_{u,i} \mid u \in U, i \in I\}$, where $U = \{u_1, \ldots, u_m\}$ is the set of genuine users, and $I = \{i_1, \ldots, i_n\}$ is the set of items. Each $D_{u,i} \in \mathcal{V}$ denotes the interaction between user u and item i, where $\mathcal{V}$ is the domain of interaction values (*e.g.*, numerical ratings $\mathbb{R}$ or textual reviews $\mathcal{T}$).

An attacker introduces a set of fake users U_a, and constructs corresponding fake interaction data $\mathcal{D}_a = \{D_{u_a,i} \mid u_a \in U_a, i \in I\}$. The manipulated input to the system becomes $\mathcal{D}' = \mathcal{D} \cup \mathcal{D}_a$.

The attacker aims to manipulate the recommendation algorithm $\Phi : \mathcal{D} \rightarrow \mathcal{O}$, where $\mathcal{O}$ denotes the system output (*e.g.*, predicted rating matrix or top-N list), to significantly alter the recommendation performance of the target item $i_t \in I$.

In the case of a push attack, the attacker aims to improve the recommendation outcome for the target item, defined as:

$$\Delta\Phi(i_t) = \Phi(\mathcal{D}')_{i_t} - \Phi(\mathcal{D})_{i_t} \tag{1}$$

Here, $\Phi(\mathcal{D})i_t$ and $\Phi(\mathcal{D}')i_t$ denote the recommendation outcome for item i_t before and after the attack, respectively (*e.g.*, predicted score or ranking position).

The shilling attack can be formulated as the following optimization problem:

$$\max_{U_a,\, \mathcal{D}_a} \Delta\Phi(i_t) \tag{2}$$

3.2 Anatomy of Shilling Attacks

The interacted items of each fake user $u_a \in U_a$ are typically partitioned into three categories, each governed by a different distribution determined by the attack method.

Target Items. For the target item i_t, fake user interactions are generated to follow a biased distribution P_{target} reflecting the attacker's adversarial intent:

$$D_{U_a,i_t} \sim P_{target} \tag{3}$$

For example, in a push attack, fake users may assign high ratings and generate positive reviews for the target item. In a nuke attack, they may assign low ratings and post negative reviews.

Selected Items. Beyond the target item, the attacker typically selects a set of items $I_s \subseteq I \setminus \{i_t\}$ to emulate genuine user behavior. For each selected item $i_s \in I_s$, fake user interactions are generated to follow a distribution:

$$D_{U_a,i_s} \sim P_{selected} \tag{4}$$

Here, $P_{selected}$ is designed to approximate typical user behavior and improve the stealth of fake profiles. For instance, in a bandwagon attack, fake users may assign high ratings to popular items to resemble normal user patterns.

Filler Items. The filler item set is defined as $I_f \subseteq I \setminus (\{i_t\} \cup I_s)$. For each filler item $i_f \in I_f$, the fake user's interaction is generated to follow:

$$D_{U_a,i_f} \sim P_{filler} \tag{5}$$

where P_{filler} is defined according to the specific attack method. The filler set may vary across fake users to increase interaction diversity and reduce detectability.

The remaining items, with which the fake user does not interact, are denoted as $I_\phi = I \setminus (i_t \cup I_s \cup I_f)$.

In essence, different shilling attack methods differ in how they construct U_a, choose I_s and I_f, and define the corresponding interactions D_{U_a,i_t}, D_{U_a,i_s}, and D_{U_a,i_f}. These components are collectively

optimized to maximize $\Delta\Phi(i_t)$. Equally important is ensuring that fake user behavior is well-camouflaged within the distribution of genuine users, thereby enhancing their resistance to detection.

4 Methodology

This section details the design of Agent4SR, which employs an LLM-based user agent with a modular architecture comprising a **profile**, **memory**, and **action** module [47, 55]. These components are integrated to support targeted and effective shilling attacks. The method is structured around three core design components:

(1) Adversarial profile construction via target-guided inference, followed by validation and diversity enhancement.
(2) Hybrid memory retrieval based on relevance and recency.
(3) Multifaceted interaction attack enhanced by target feature propagation strategy.

The overall framework is illustrated in Figure 2.

4.1 Modular Structure of Fake User Agent

We formally represent the fake user agent as a triplet Agent = $(\mathcal{P}rofile, \mathcal{M}emory, \mathcal{A}ction)$, comprising three core modules:

- **Profile Module**: Stores personalized attributes that define the agent's identity and preferences, such as age, gender, occupation, user-specific interests, and interaction habits (*e.g.*, rating or reviewing frequency).
- **Memory Module**: Stores the agent's historical interactions, including ratings and reviews. It employs two retrieval methods: a relevance-based retriever and a recency-based retriever, to distill key information and support decision making.
- **Action Module**: Defines the agent's behavior, including item selection, rating generation, and review generation.

4.2 Target-Aware Profile Construction

To construct a set of fake user agents whose preferences align with a given target item, we propose a target-driven method that reversely infers user profiles from the target item's characteristics, followed by validation and diversification.

4.2.1 Target Item Feature Analysis. Given a target item i_t, we construct a prompt containing its basic metadata (*e.g.*, title and category), and invoke the LLM to generate an analysis of its characteristics $\mathcal{A}_t = \text{Prompt}_{\text{LLM}}(i_t)$ based on its pretrained world knowledge.

4.2.2 Profile Inference from Target Item. Based on the item analysis $\mathcal{A}_t$ and the attack type $d \in \{+, -\}$ (push or nuke), we construct the prompt for profile inference. For example, in a push attack on the item titled "*High-Definition Large-Size Electronic Display Screen*", the prompt is constructed as:

Example Prompt for Profile Inference

You currently have a strong preference for High-Definition Large-Size Electronic Display Screen.
The information about this item is as follows: It features an ultra-large 32-inch curved design with high resolution and exceptional clarity. It is suitable for gaming, multitasking, office work, and watching TV or other media.
Based on this item description, please infer your potential personality profile, including:

- **Gender, age, and occupation**
- **Personal interests or dislikes**
- **Interaction habits** (*e.g.*, **how frequently you provide reviews or ratings**)

The LLM output from this prompt defines the Profile module $\mathcal{P}^{(i_t)}$, comprising user interests, interaction habits, and other personality traits.

$$\mathcal{P}^{(i_t)} = Prompt_{\text{LLM}}(\mathcal{A}_t, d) \tag{6}$$

4.2.3 Profile Consistency Validation. To verify that the generated profile $\mathcal{P}^{(i_t)}$ logically aligns with the target item i_t, we introduce a validation step inspired by the encoding-decoding architecture in Seq2Seq models [45]. The profile generation serves as the encoding phase, while validation corresponds to decoding. Specifically, we issue a secondary prompt asking the fake user agent, based on its current profile, whether it would genuinely prefer the target item, as in the case of a push attack. A negative response (*i.e.*, $\text{Validate}(\mathcal{P}^{(i_t)}) = 0$) triggers profile regeneration to ensure consistency with the attack objective.

$$\text{Validate}(\mathcal{P}^{(i_t)}) = \begin{cases} 1, & \text{if } Prompt_{\text{LLM}}(\mathcal{P}^{(i_t)}, i_t, d)\text{=Yes} \\ 0, & \text{otherwise} \end{cases} \tag{7}$$

4.2.4 Profile Diversity Enhancement. To enhance diversity among fake user agents and reduce the risk of detection caused by overly concentrated interaction patterns, we leverage the LLM to select a subset of profiles that maximizes diversity from the candidate pool:

$$\{\mathcal{P}_k^{(i_t)}\}_{k=1}^{N} = Prompt_{\text{LLM}}(\{\mathcal{P}_k^{(i_t)}\}_{k=1}^{M}) \tag{8}$$

where M is the number of candidate profiles generated, and N is the number of fake user agents to be injected.

Overall, in a push attack, each fake user agent is individually crafted to resemble a genuine user who appears to hold authentic preferences for the target item. At the population level, the fake users exhibit diversity comparable to real-world user distributions. When interacting with the RS, their behaviors appear highly plausible and are difficult to detect.

4.3 Memory Management and Hybrid Retrieval

The Memory module stores the fake user agent's historical interactions, consisting of rating records $\mathcal{M}_R = (i_m, \mathbb{R}_m, t_m)$ and review records $\mathcal{M}_T = (i_n, \mathcal{T}_n, t_n)$, where i denotes the item, $\mathbb{R}$ the rating, $\mathcal{T}$ the review, and t the timestamp. The module supports memory updates and retrieval to guide downstream decisions.

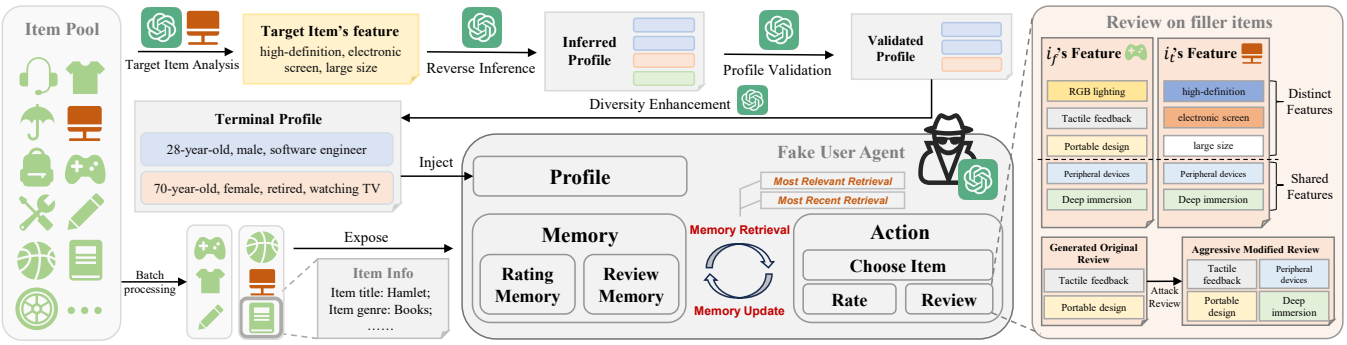

Figure 2: The overall framework of Agent4SR. It starts with target item analysis and profile inference, followed by profile validation and diversification. Fake user agents interact with filler items using hybrid memory retrieval, while a target feature propagation strategy embeds key target attributes into reviews on filler items.

4.3.1 Memory Update Mechanism. Upon executing an action, the agent updates its memory by recording the new rating $(i_q, \mathbb{R}_{new}, t_{now})$ in $\mathcal{M}_R$ and the corresponding review $(i_q, \mathcal{T}_{new}, t_{now})$ in $\mathcal{M}_T$. To avoid excessive memory accumulation, only a fixed number of recent entries may be retained.

4.3.2 Memory Retrieval Mechanism. Given a query item i_q, we perform two types of retrieval: relevance-based and recency-based.

(1) **Relevance-based retrieval** selects the top-K semantically relevant entries from the full memory set $\mathcal{M} = \mathcal{M}_R \cup \mathcal{M}_T$:

$$\mathcal{M}_{rel} = \{m_j \in \mathcal{M} \mid sim(i_q, i_j) \geq Top_K(\{sim(v_q, v_k)\}_{k=1}^{|\mathcal{M}|})\} \quad (9)$$

where sim$(\cdot)$ denotes semantic similarity based on item information. Specifically, item semantics are extracted using a Transformer-based language model, which encodes textual metadata (*e.g.*, item titles) into dense vector representations. Cosine similarity is then computed between these embeddings.

(2) **Recency-based retrieval** selects the M most recent interaction records:

$$\mathcal{M}_{rec} = \{m_i \in \mathcal{M} \mid t_i \geq Top_M(\{t_j\}_{j=1}^{|\mathcal{M}|})\} \quad (10)$$

In summary, the final retrieved memory set $\mathcal{M}_{ret} = \mathcal{M}_{rel} \cup \mathcal{M}_{rec}$ integrates relevance and recency to preserve the agent's core interests and recent behavioral patterns, effectively emulating human memory retrieval processes.

4.4 Multifaceted Rating-Review Attack

For the target item, each fake user agent u_a generates a rating and textual review as adversarial interaction data D_{u_a,i_t}. In a push attack, this typically involves assigning the highest rating and a positive review.

Agent4SR does not explicitly define a selected item set I_s. Instead, for remaining items, each fake user agent individually selects items in a personalized manner, forming a filler item set I_f, and generates corresponding interaction data D_{u_a,I_f}. In each exposure batch, the fake user agent executes a three-step process: item selection, rating generation, and review generation.

4.4.1 Item Selection. For each exposure batch $B^{(b)} = i_1, \ldots, i_k$ in round b, the fake user agent selects a subset $I_{f,b} \subseteq B^{(b)}$ as filler items, guided by its constructed malicious profile $\mathcal{P}_{u_a}$. This process is repeated over multiple rounds, and the final filler item set I_f is obtained by aggregating filler items from all exposure rounds:

$$I_f = \bigcup_{b=1}^{\mathbb{N}} Prompt_{\text{LLM}}(B^{(b)}, \mathcal{P}_{u_a}) \quad (11)$$

where $\mathbb{N}$ is the total number of exposure rounds.

This approach offers two key advantages:

(1) In terms of the number of filler items, Agent4SR provides only a flexible reference, allowing each fake user to determine the extent of engagement based on individual interaction habits embedded in $\mathcal{P}_{u_a}$. In contrast to conventional methods that prescribe a fixed number of interactions for each fake user, Agent4SR promotes behavioral diversity.

(2) In terms of the composition of the filler item set, unlike conventional attack methods that rely on random choice or simple heuristics, Agent4SR conditions item selection on the fake user's stated preferences. This results in filler items that are more consistent with the user profile, yielding more coherent interaction behavior with improved stealth.

4.4.2 Rating Generation. The fake user agent is prompted to rate the filler items I_f based on its profile and the memory retrieved via the previously described hybrid mechanism, ensuring alignment with its stated preferences and interaction history:

$$\mathbb{R}(u_a, I_f) = Prompt_{\text{LLM}}(I_f, \mathcal{P}_{u_a}, \mathcal{M}_{ret}) \quad (12)$$

This design supports profile-aligned and historically coherent rating behavior, enhancing the human-like characteristics of agent behavior.

4.4.3 Adversarial Review Generation via Target Feature Propagation. We propose a review-based attack strategy that propagates prominent features of the target item across filler items. For each filler item, the agent first identifies shared characteristics with the target item, guided by its analysis result $\mathcal{A}_t$ obtained during the earlier profile construction stage. It then generates an initial review $\widetilde{\mathcal{T}}(u_a, I_f)$ conditioned on the item's attributes and the rating $\mathbb{R}(u_a, I_f)$, and incorporates the identified similarity $S(I_f, i_t)$ to produce the final review $\mathcal{T}(u_a, I_f)$.

$$S(I_f, i_t) = Prompt_{\text{LLM}}(\mathcal{A}_t, I_f) \quad (13)$$

$$\widetilde{\mathcal{T}}(u_a, I_f) = Prompt_{\text{LLM}}(I_f, \mathbb{R}(u_a, I_f)) \tag{14}$$

$$\mathcal{T}(u_a, I_f) = Prompt_{\text{LLM}}(S(I_f, i_t), \widetilde{\mathcal{T}}(u_a, I_f)) \tag{15}$$

For example, when reviewing the filler item *Phantom X Ultra-Performance Portable Gaming Controller*, the fake user agent first generates an initial review based on the item's attributes. After identifying similarities with the target item (*e.g.*, "both are peripheral devices" and "both offer deep immersion"), it revises the review to incorporate these shared characteristics. The transformation is illustrated below:

Example Review Before and After Feature Propagation

Before Propagation:
"This is easily the best controller I've used. The tactile feedback is just right, and it handles tough gameplay without a hitch. The portable design makes it super easy to carry around. Great value overall."

After Propagation:
"As one of the top **peripheral devices** I've tried, this controller delivers excellent tactile feedback and easily handles demanding gameplay. It offers a **deeply immersive** experience, and the portable design makes it perfect for gaming on the go."

Through this process, the target item's features are implicitly propagated to other items. If users engage with these reviewed items, the RS may infer a latent interest in the target item. This manipulation can bias predicted ratings and increase the target item's exposure.

To summarize, following the above procedure, the fake user agent derives the interaction data for the filler items as:

$$(\mathbb{R}(u_a, I_f), \mathcal{T}(u_a, I_f)) \sim P_{filler} \mid I_f, \mathcal{P}_{u_a}, \mathcal{M}_{ret}, \mathcal{A}_t \tag{16}$$

4.5 Agent4SR Execution Pipeline

The overall attack procedure is summarized in Algorithm 1, comprising the following stages:

Initialization. Specify the target item i_t and the attack type $d \in \{+, -\}$, construct N fake user agents $\{Agent_i\}_{i=1}^{N}$, initialize their profile modules as described in the profile construction section, and set the number of exposure rounds $\mathbb{N}$.

Adversarial Target Item Interaction. Each fake agent interacts with the target item i_t, generating either a rating or a review according to the attack type d.

Iterative Filler Item Interaction. In each round b ($1 \leq b \leq \mathbb{N}$), the victim RS exposes a batch of items $B^{(b)}$ to a fake user agent u_a. Guided by its profile, the agent selects a filler item subset $I_{f,b} \subseteq B^{(b)}$, retrieves memory via the hybrid retrieval mechanism, and generates corresponding ratings $\mathbb{R}(u_a, I_{f,b})$ and reviews $\mathcal{T}(u_a, I_{f,b})$. The resulting interactions are recorded in the memory module to support subsequent rounds.

By simulating the complete process through which real users browse, engage with, and evaluate items in RS, Agent4SR constructs fake users agents with realistic behaviors that improve both stealth and attack effectiveness.

Algorithm 1 Execution Procedure of Agent4SR

Require: Target item i_t; attack type d; number of fake users N; number of exposure rounds $\mathbb{N}$
Ensure: Interaction data $\{D_{u_a,i_t}, D_{u_a,I_f}\}_{a=1}^{N}$

```
Initialization:
Construct N fake user agents and initialize their profiles based on i_t and d (Eq. 6, 7, 8)
Target Item Interaction:
for each agent u_a do
    Generate rating or review on i_t according to d to form D_{u_a,i_t}
end for
Iterative Filler Item Interaction:
for b = 1 to ℕ do
    for each agent u_a do
        Receive exposure batch B^(b) from victim RS
        Select filler items I_{f,b} ⊆ B^(b) based on profile (Eq. 11)
        Retrieve memory using hybrid strategy (Eq. 9, 10)
        Generate rating or review for I_{f,b} to construct D_{u_a,I_{f,b}} (Eq. 12, 13, 14, 15)
        Update memory
    end for
end for
Return {D_{u_a,i_t}, D_{u_a,I_f}}_{a=1}^N where D_{u_a,I_f} = ⋃_{b=1}^ℕ D_{u_a,I_{f,b}}
```

Table 1: Statistics of the datasets.

Datasets	#Users	#Items	#Inters.	Sparsity
Books	3,441	3,569	41,764	99.66%
Auto	2,700	3,090	29,346	99.64%
Pets	880	522	5,906	98.71%

5 Experiment

5.1 Settings

5.1.1 Datasets. We conducted experiments on three Amazon datasets: *Books*, *Automotive* (*Auto*), and *Pet Supplies* (*Pets*), each containing both rating and review data. Agent4SR was implemented with GPT-4o-mini. Due to API cost constraints, we limited the interaction data to a one-year period. To mitigate the impact of cold-start users and items, we retained only those with at least five interactions. The resulting dataset size and sparsity are broadly consistent with prior studies on shilling attacks [23, 24, 54], as shown in Table 1. The data were randomly split into training, validation, and test sets in an 8:1:1 ratio to ensure sufficient training of the victim RS.

5.1.2 Attack Configuration and Victim Models. By default, we inject fake users equivalent to 5% of the real user base, with the push attack as the representative scenario. We randomly select 10 items from the item pool as target items to evaluate average attack performance. To ensure broad coverage, we evaluate Agent4SR on two types of victim RS: those based solely on ratings and those incorporating both ratings and reviews. For rating-based RS, we adopt NMF [20] and NeuNMF [9]. For models integrating reviews and ratings, we

use two dual-tower variants: Dual-Tower$_{early}$ and Dual-Tower$_{late}$, which integrate review and ID features via early and late fusion, respectively [14]. The exposure mechanism in victim RS follows a popularity-based probabilistic scheme. While item visibility is determined by this mechanism, Agent4SR does not rely on access to popularity data and assumes exposure is controlled by the RS. Since online exposure cannot be directly simulated in offline settings, we approximate the exposure process using a popularity-driven mechanism.

5.1.3 Evaluation Metrics. We adopt metrics aligned with [23] to evaluate the effectiveness of Agent4SR, including prediction shift and hit ratios at K (HR@K), where K = 10 in our experiments. Predicion shift measures the absolute change in the predicted rating of the target item before and after the attack. HR@K reflects the relative ranking of the target item by indicating how frequently it appears among the top-K recommendations. To assess the attack's effect on the victim RS's predictive accuracy, we report RMSE and MAE as indirect indicators of stealthiness, measuring whether the attack leads to noticeable degradation in prediction performance.

5.1.4 Baselines. As a black-box attack that operates without access to internal RS data (*e.g.*, the rating matrix or RS feedback), Agent4SR is compared with conventional attacks and existing low-knowledge attack baselines, all of which assign the maximum rating to the target item:

- **Random Attack** [19]: Each filler item is assigned a rating $\mathbb{R}(u_a, i_f) \sim \mathcal{N}(\mu, \sigma)$, where μ and σ denote the mean and standard deviation computed over all ratings in the system.
- **Average Attack** [19]: Each filler item is assigned a rating $\mathbb{R}(u_a, i_f) \sim \mathcal{N}(\mu, \sigma)$, where μ and σ denote the mean and standard deviation of all ratings on item i_f in the system.
- **Bandwagon Attack** [35]: The attack profile includes the most popular items as selected items, each assigned the maximum rating. Filler items are randomly selected and rated following the same strategy as in Random Attack.
- **Segmented Attack** [3]: This method targets a user subgroup likely to engage with the target item via strategically selected items. Due to data limitations, the most popular items are used as selected items and assigned the maximum rating, while filler items are randomly chosen and assigned the minimum rating.
- **Average over Popular** (AoP) [16]: This technique is designed to obscure average attacks. Filler items are uniformly selected from the top $\mathcal{X}$% of items by popularity. in our experiments, we set $\mathcal{X} = 10$.
- **Mixed Attack** [1]: This strategy integrates Random, Average, Bandwagon, and Segmented attacks in equal proportion.
- **R-Tojan(review)** [54]: A review-aware attack method uses a fine-tuned LLM to generate reviews conditioned on ratings. As its rating manipulation relies on the rating matrix, we adopt only its review generation module to augment the baselines.

5.2 Overall Performance

We evaluated Agent4SR on all three datasets and compared its performance with baseline methods. Tables 2 and 3 present attack results on rating-based and review-aware RS, respectively. The key observations are as follows:

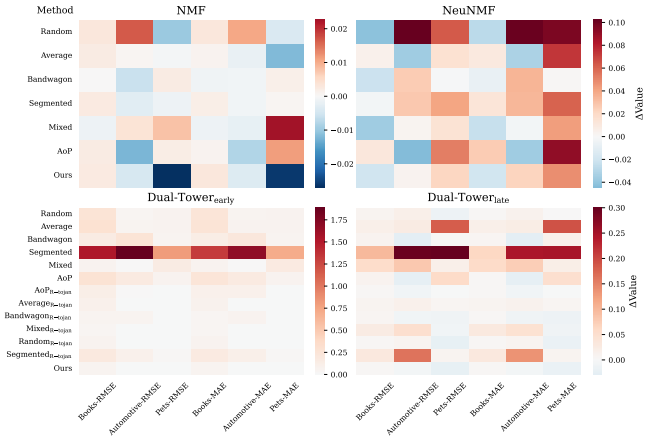

Figure 3: Heatmaps of RMSE/MAE shift (ΔValue) across attacks and datasets. Lighter colors indicate smaller shifts. Red denotes increased RMSE/MAE (performance drop), blue denotes decreased RMSE/MAE (performance gain). Scales are normalized per subplot.

(1) Agent4SR consistently outperforms most baselines, demonstrating the effectiveness of using LLM-based agents to simulate users in shilling attacks. In **rating-only RS**, it achieves notably superior performance in both prediction shift and HR@10, surpassing most conventional attacks. While some methods may occasionally outperform Agent4SR, they fail to match the robustness exhibited by Agent4SR across varied settings. Extending to **review-aware RS**, we adapt rating-based attacks with synthetic reviews for fair comparison. One variant generates review embeddings using random vectors that mimic real distributions, while the other produces textual reviews via R-Tojan-style generation. Our adversarially crafted reviews result in significantly higher attack effectiveness, highlighting the strength of Agent4SR in more complex, real-world recommendation scenarios.

(2) Among all datasets, Agent4SR achieves the strongest attack performance on the Books dataset, consistently outperforming all baselines across evaluation metrics. This may be attributed to the LLM-based agent's broader domain knowledge in books, which enables it to more effectively perform key reasoning steps such as inferring user profiles and analyzing item characteristics, ultimately leading to stronger adversarial behavior.

(3) We evaluate the changes in RMSE and MAE of the victim RS under attack, as illustrated in Figure 3. Agent4SR consistently results in minor impact on prediction accuracy, indicating limited interference with the RS and highlighting the stealthiness of Agent4SR. In rating-only RS, most attack methods cause only limited degradation in RMSE and MAE. In contrast, Segmented Attack leads to substantial performance drop in review-aware RS, likely due to its use of a semantically narrow item set and extreme ratings for both selected and filler items, which impairs model generalization. Notably, in review-aware RS, conventional attacks cause considerably less disruption to RMSE and MAE when combined with R-Tojan-style reviews, consistent with the findings in [54].

Table 2: Overall performance on rating-only RS. Best results are highlighted in bold.

Victim RS	NMF						NeuNMF					
Method	Prediction Shift			HR@10			Prediction Shift			HR@10		
	Books	Auto	Pets	Books	Auto	Pets	Books	Auto	Pets	Books	Auto	Pets
Random	0.6491	0.2828	0.9866	0.0997	0.0121	0.0696	0.8662	0.4979	1.0201	0.0414	0.0128	0.1496
Average	0.6191	0.2736	0.9773	0.0693	0.0054	0.0502	0.8336	0.4872	0.9411	0.0425	0.0070	0.1202
Bandwagon	0.6187	0.2783	0.9397	0.0726	0.0084	0.0433	0.7262	0.4546	0.9581	0.0286	0.0107	0.1162
Segmented	0.6303	0.3127	0.8689	0.1886	0.0320	**0.1160**	0.4669	0.1280	0.4326	0.0202	0.0115	0.0472
Mixed	0.6175	0.2691	0.8924	0.0670	0.0084	0.0393	0.8335	0.4683	0.9396	0.0337	0.0142	0.1575
AoP	0.6400	0.2818	0.9816	0.0813	0.0085	0.0674	0.8828	0.4756	**1.0486**	0.0438	0.0135	0.1464
Agent4SR	**0.7366**	**0.3421**	**1.0327**	**0.1891**	**0.0618**	0.0934	**0.9064**	**0.5020**	1.0371	**0.0548**	**0.0225**	**0.1885**

Table 3: Overall performance on review-aware RS. Best results are highlighted in bold.

Victim RS	Dual-Tower$_{early}$						Dual-Tower$_{late}$					
Method	Prediction Shift			HR@10			Prediction Shift			HR@10		
	Books	Auto	Pets	Books	Auto	Pets	Books	Auto	Pets	Books	Auto	Pets
Random	0.0021	0.1297	0.1966	0.0041	0.0024	0.0212	0.0022	0.0005	0.0375	0.0040	0.0021	0.0192
Average	0.0064	-0.0179	0.0491	0.0028	0.0012	0.0099	0.0445	0.0043	0.0458	0.0050	0.0017	0.0181
Bandwagon	0.0002	0.0332	0.0585	0.0026	0.0023	0.0107	0.0032	0.0119	0.0007	0.0033	0.0029	0.0147
Segmented	0.0035	0.1732	0.2207	0.0056	0.0012	0.0205	0.0369	0.0075	0.0172	0.0054	0.0024	0.0144
Mixed	0.1586	0.0351	0.2114	0.0060	0.0011	0.0179	0.0295	0.0287	0.0165	0.0032	0.0030	0.0200
AoP	0.0074	0.0133	0.0913	0.0030	0.0014	0.0124	0.0633	-0.0160	0.0253	0.0043	0.0031	0.0180
Random$_{R\text{-}Tojan}$	0.0062	0.1304	0.2015	0.0046	0.0027	0.0232	0.0058	0.0179	0.0477	0.0046	0.0026	0.0198
Average$_{R\text{-}Tojan}$	0.0136	0.0526	0.0593	0.0032	0.0012	0.0165	0.0632	0.0063	0.0512	0.0055	0.0023	0.0192
Bandwagon$_{R\text{-}Tojan}$	0.0037	0.0578	0.0634	0.0033	0.0028	0.0151	0.0052	0.0268	0.0069	0.0044	0.0030	0.0156
Segmented$_{R\text{-}Tojan}$	0.0051	0.1803	0.2423	0.0064	0.0014	0.0221	0.0516	0.0217	0.0258	0.0057	0.0026	0.0154
Mixed$_{R\text{-}Tojan}$	0.1693	0.0729	0.2234	0.0074	0.0014	0.0185	0.0378	0.0411	0.0354	0.0034	0.0030	0.0205
AoP$_{R\text{-}Tojan}$	0.0112	0.0136	0.1228	0.0046	0.0016	0.0136	0.0966	0.0103	0.0287	0.0049	**0.0033**	0.0185
Agent4SR	**0.1967**	**0.2274**	**0.2660**	**0.0112**	**0.0032**	**0.0269**	**0.1205**	**0.0551**	**0.0651**	**0.0063**	0.0031	**0.0210**

5.3 Detection Evasion Analysis

To further validate the stealthiness of Agent4SR, we adopt two advanced fake user detection approaches: one unsupervised and one supervised [53, 59]. Precision and recall are reported for identifying the injected users. As shown in Figure 4, both methods exhibited the weakest performance when applied to users created by Agent4SR. This suggests that our fake users, driven by the LLM-based agent's human-like reasoning and generation capabilities, are substantially harder to detect due to their ability to closely mimic genuine user behavior.

Due to the scarcity of shilling attack methods that incorporate reviews, most existing fake user detection techniques are primarily designed for user rating data. The detectability of LLM-generated text largely depends on the model's reasoning and generation capabilities, and is therefore beyond the scope of this study.

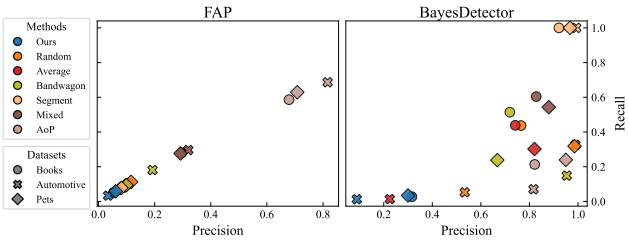

Figure 4: Precision-Recall results under two detectors. Lower values indicate better stealthiness of the attack.

5.4 Further Model Analysis

5.4.1 Ablation study. Agent4SR integrates six key components to construct realistic and deceptive fake users: (1) Profile inference and validation; (2) Profile diversity enhancement; (3) Relevance-based memory retrieval; (4) Recency-based memory retrieval; (5) Review-based attack; (6) Target-aware feature propagation. To assess their individual contributions, we perform ablation experiments on the Books dataset by disabling each component in turn while keeping the rest intact, targeting two review-aware RS. Results are reported in Table 4.

(1) *w/o* Profile inference and validation: In this variant, the agent's profile is randomly assigned rather than inferred and validated based on the target item. The agent still assigns the highest

Table 4: Ablation results on the Books dataset. For brevity, only HR@10 and RMSE are shown for Dual-Tower$_{early}$ and Dual-Tower$_{late}$, reflecting effectiveness and stealth. Underlined values indicate significant performance degradation.

Method Variant	Dual-Tower$_{early}$		Dual-Tower$_{late}$	
	HR@10	RMSE Shift	HR@10	RMSE Shift
Full Model (Agent4SR)	**0.0112**	**0.0581**	**0.0063**	**0.0042**
w/o Profile inference and validation	0.0078	0.0578	0.0044	0.0042
w/o Profile diversity enhancement	0.0108	0.0623	0.0066	0.0052
w/o Relevance-based memory retrieval	0.0116	0.0676	0.0058	0.0062
w/o Recency-based memory retrieval	0.0092	0.0569	0.0052	0.0044
w/o Review-based attack	0.0072	0.0679	0.0058	0.0054
w/o Target-aware feature propagation	0.0078	0.0575	0.0056	0.0040

rating or a positive review to the target. Although the RMSE shift shows only a slight decrease, the attack performance drops substantially.

(2) *w/o* Profile diversity enhancement: In this variant, diversity enhancement is disabled after profile generation. The increased RMSE shift indicates reduced stealthiness, while attack performance remains largely unaffected.

(3) *w/o* Relevance-based memory retrieval: In this setting, the memory module retrieves only the most semantically relevant past interactions. The attack performance remains relatively stable, while stealthiness degrades significantly.

(4) *w/o* Recency-based memory retrieval: In this setting, the memory module retrieves only the most recent interactions. Stealthiness remains largely unchanged, but the attack performance deteriorates.

(5) *w/o* Review-based attack: Here, when attacking review-aware RS, we omit review-based attack and rely solely on ratings. For review representations, we adopt the same approach as in baseline methods by initializing review embeddings with random vectors that mimic the distribution of real ones. The results show that both attack effectiveness and stealthiness decline significantly under this configuration.

(6) *w/o* Target-aware feature propagation: Here, the LLM-based agent generates reviews directly based on its profile, item information, and interaction rating, without employing our proposed review attack strategy. We observe a drop in attack performance, whereas the RMSE shift shows only a slight decrease.

Overall, the ablation results demonstrate that each component in Agent4SR contributes meaningfully to either enhancing attack effectiveness or improving stealthiness, with several components supporting both.

5.4.2 Effect of Injection Size on Attack Strength. We evaluate the impact of attack size on the victim RS by injecting different proportions of fake users. As shown in Figure 5, increasing the proportion of injected users significantly amplifies the attack effect, as the target item is recommended more frequently to real users.

5.4.3 Vulnerability of Low-Activity Users. Prior work [8, 41] has shown that users in RS exhibit varying levels of vulnerability to shilling attacks. To investigate this further, we analyze the impact of Agent4SR on low-activity users by comparing the bottom 20% of genuine users (by interaction count) against the overall user population. As shown in Figure 6, users with fewer interactions

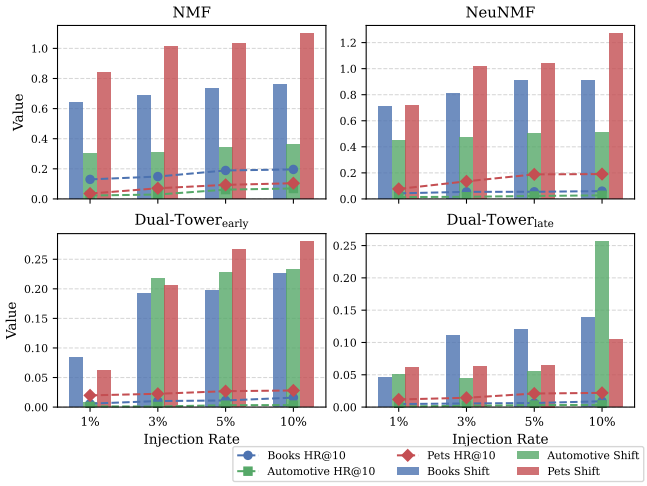

Figure 5: Attack effectiveness at different injection rates. Each subfigure corresponds to a victim RS.

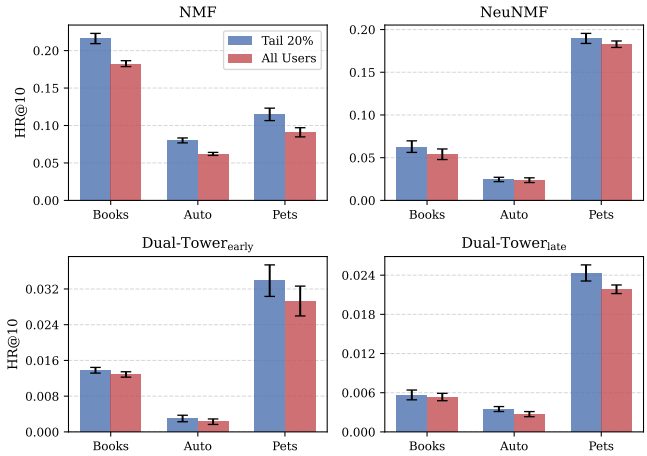

Figure 6: Comparison of Agent4SR's impact on low-activity users (bottom 20%) vs. overall population.

tend to achieve higher HR@10 for the target item after the attack, indicating greater susceptibility. This is likely due to the system's limited ability to accurately model the preferences of low-activity users, making them more easily influenced.

In real-world RS, low-activity users are at greater risk of churn. Their weaker commitment makes them more vulnerable to manipulated recommendations, as confirmed by our results. These findings suggest that the potential harm of Agent4SR may be greater than initially expected.

5.4.4 Vulnerability of Long-Tail Items. Push attacks typically target low-popularity items, as popular items seldom require further promotion. To reflect this practical setting, we select target items from the bottom 20% of the popularity distribution and compare them with randomly selected targets. As shown in Figure 7, Agent4SR performs significantly better when attacking low-popularity items.

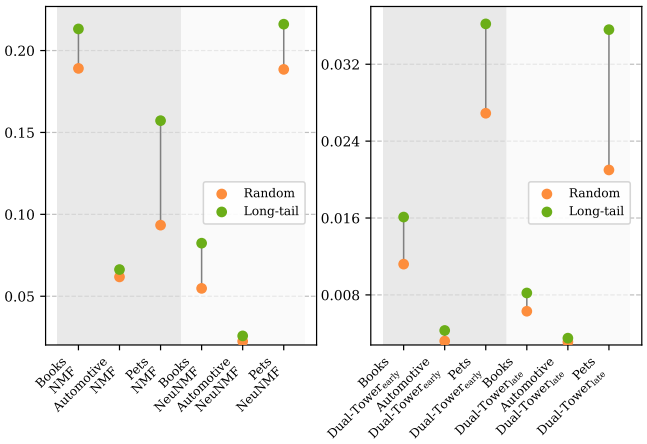

Figure 7: Comparison of HR@10 when targeting long-tail vs. randomly selected items. Each group shows the performance of a victim RS on different datasets.

This is likely because sparse interactions in RS make low-popularity items highly sensitive to even a few injected positives. As a result, push attacks substantially boost the rank and exposure of long-tail items, while popular items remain more resistant due to abundant historical signals. These findings highlight the heightened risk Agent4SR poses in realistic attack scenarios.

6 Conclusion and Future work

In this paper, we propose Agent4SR, a shilling attack framework that leverages LLM-based agents to simulate fake users and carry out rating- and review-based attacks on RS. The proposed method outperforms existing low-knowledge baselines in both effectiveness and stealth.

We also observe that Agent4SR is most effective on the Books dataset, likely due to LLMs' richer domain knowledge in this area. This suggests that further enhancing LLM capabilities, such as retrieval-augmented generation or fine-tuning [22, 37], may amplify attack performance even further.

While our evaluation focuses on e-commerce scenarios, the potential consequences may be even more pronounced in **U**ser-**G**enerated **C**ontent (UGC) platforms. In such settings, manipulated recommendations can mislead content creators, prompting them to produce content that aligns with fabricated user interests [2, 18]. Over time, this may cause thematic drift, reduce content diversity, and ultimately lead to user disengagement and community decline. As LLM-based agents continue to evolve, the emergence of virtual users in online communities becomes an increasingly tangible reality. We hope this work draws greater attention to their security implications in RS and encourages future research on understanding broader attack surfaces and developing robust defense strategies.

References

[1] Runa Bhaumik, Bamshad Mobasher, and Robin Burke. 2011. A clustering approach to unsupervised attack detection in collaborative recommender systems. In *Proceedings of the International Conference on Data Science (ICDATA)*. Citeseer, 1.

[2] Craig Boutilier, Martin Mladenov, and Guy Tennenholtz. 2023. Modeling recommender ecosystems: Research challenges at the intersection of mechanism design, reinforcement learning and generative models.

[3] Robin Burke, Bamshad Mobasher, and Runa Bhaumik. 2005. Limited knowledge shilling attacks in collaborative filtering systems. In *Proceedings of 3rd international workshop on intelligent techniques for web personalization (ITWP 2005), 19th international joint conference on artificial intelligence (IJCAI 2005)*. 17–24.

[4] Shihao Cai, Jizhi Zhang, Keqin Bao, Chongming Gao, Qifan Wang, Fuli Feng, and Xiangnan He. 2025. Agentic Feedback Loop Modeling Improves Recommendation and User Simulation. arXiv:2410.20027 [cs.IR] https://arxiv.org/abs/2410.20027

[5] Hung-Yun Chiang, Yi-Syuan Chen, Yun-Zhu Song, Hong-Han Shuai, and Jason S Chang. 2023. Shilling black-box review-based recommender systems through fake review generation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 286–297.

[6] Konstantina Christakopoulou and Arindam Banerjee. 2018. Adversarial recommendation: Attack of the learned fake users. *arXiv preprint arXiv:1809.08336* (2018).

[7] Konstantina Christakopoulou and Arindam Banerjee. 2019. Adversarial attacks on an oblivious recommender. In *Proceedings of the 13th ACM Conference on Recommender Systems*. 322–330.

[8] Yashar Deldjoo, Tommaso Di Noia, and Felice Antonio Merra. 2019. Assessing the impact of a user-item collaborative attack on class of users. *arXiv preprint arXiv:1908.07968* (2019).

[9] Gintare Karolina Dziugaite and Daniel M Roy. 2015. Neural network matrix factorization. *arXiv preprint arXiv:1511.06443* (2015).

[10] Luke Friedman, Sameer Ahuja, David Allen, Zhenning Tan, Hakim Sidahmed, Changbo Long, Jun Xie, Gabriel Schubiner, Ajay Patel, Harsh Lara, et al. 2023. Leveraging large language models in conversational recommender systems.

[11] Jingyue Gao, Yang Lin, Yasha Wang, Xiting Wang, Zhao Yang, Yuanduo He, and Xu Chu. 2020. Set-sequence-graph: A multi-view approach towards exploiting reviews for recommendation. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 395–404.

[12] Ian J Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative adversarial nets.

[13] Mingzhe Han, Dongsheng Li, Jiafeng Xia, Jiahao Liu, Hansu Gu, Peng Zhang, Ning Gu, and Tun Lu. 2025. FedCIA: Federated Collaborative Information Aggregation for Privacy-Preserving Recommendation. *arXiv preprint arXiv:2504.14208* (2025).

[14] Po-Sen Huang, Xiaodong He, Jianfeng Gao, Li Deng, Alex Acero, and Larry Heck. 2013. Learning deep structured semantic models for web search using clickthrough data. In *Proceedings of the 22nd ACM international conference on Information & Knowledge Management*. 2333–2338.

[15] Xu Huang, Jianxun Lian, Yuxuan Lei, Jing Yao, Defu Lian, and Xing Xie. 2023. Recommender ai agent: Integrating large language models for interactive recommendations.

[16] Neil Hurley, Zunping Cheng, and Mi Zhang. 2009. Statistical attack detection. In *Proceedings of the third ACM conference on Recommender systems*. 149–156.

[17] Sunghwan Kim, Tongyoung Kim, Kwangwook Seo, Jinyoung Yeo, and Dongha Lee. 2024. Stop Playing the Guessing Game! Target-free User Simulation for Evaluating Conversational Recommender Systems.

[18] Haruka Kiyohara, Fan Yao, and Sarah Dean. 2025. Policy Design for Two-sided Platforms with Participation Dynamics.

[19] Shyong K Lam and John Riedl. 2004. Shilling recommender systems for fun and profit. In *Proceedings of the 13th international conference on World Wide Web*. 393–402.

[20] Daniel Lee and H Sebastian Seung. 2000. Algorithms for non-negative matrix factorization. *Advances in neural information processing systems* 13.

[21] Seungpil Lee, Woochang Sim, Donghyeon Shin, Wongyu Seo, Jiwon Park, Seokki Lee, Sanha Hwang, Sejin Kim, and Sundong Kim. 2024. Reasoning abilities of large language models: In-depth analysis on the abstraction and reasoning corpus. *ACM Transactions on Intelligent Systems and Technology* (2024).

[22] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. 9459–9474 pages.

[23] Chen Lin, Si Chen, Hui Li, Yanghua Xiao, Lianyun Li, and Qian Yang. 2020. Attacking recommender systems with augmented user profiles. In *Proceedings of the 29th ACM international conference on information & knowledge management (CIKM '20)*. 855–864.

[24] Chen Lin, Si Chen, Meifang Zeng, Sheng Zhang, Min Gao, and Hui Li. 2022. Shilling black-box recommender systems by learning to generate fake user profiles. *IEEE Transactions on Neural Networks and Learning Systems* 35, 1 (2022), 1305–1319.

[25] Jiahao Liu, Shengkang Gu, Dongsheng Li, Guangping Zhang, Mingzhe Han, Hansu Gu, Peng Zhang, Tun Lu, Li Shang, and Ning Gu. 2025. Enhancing Cross-Domain Recommendations with Memory-Optimized LLM-Based User Agents. *arXiv preprint arXiv:2502.13843* (2025).

[26] Jiahao Liu, Dongsheng Li, Hansu Gu, Tun Lu, Jiongran Wu, Peng Zhang, Li Shang, and Ning Gu. 2023. Recommendation unlearning via matrix correction. *arXiv preprint arXiv:2307.15960* (2023).

[27] Jiahao Liu, Dongsheng Li, Hansu Gu, Tun Lu, Peng Zhang, and Ning Gu. 2022. Parameter-free dynamic graph embedding for link prediction. *Advances in Neural Information Processing Systems* 35 (2022), 27623–27635.

[28] Jiahao Liu, Dongsheng Li, Hansu Gu, Tun Lu, Peng Zhang, Li Shang, and Ning Gu. 2023. Personalized graph signal processing for collaborative filtering. In *Proceedings of the ACM Web Conference 2023*. 1264–1272.

[29] Jiahao Liu, Dongsheng Li, Hansu Gu, Tun Lu, Peng Zhang, Li Shang, and Ning Gu. 2023. Triple structural information modelling for accurate, explainable and interactive recommendation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1086–1095.

[30] Jiahao Liu, Dongsheng Li, Hansu Gu, Peng Zhang, Tun Lu, Li Shang, and Ning Gu. 2025. Mitigating Popularity Bias in Collaborative Filtering through Fair Sampling. *arXiv preprint arXiv:2502.13840* (2025).

[31] Jiahao Liu, Yiyang Shao, Peng Zhang, Dongsheng Li, Hansu Gu, Chao Chen, Longzhi Du, Tun Lu, and Ning Gu. 2025. Filtering Discomforting Recommendations with Large Language Models. In *Proceedings of the ACM on Web Conference 2025*. 3639–3650.

[32] Jiahao Liu, Xueshuo Yan, Dongsheng Li, Guangping Zhang, Hansu Gu, Peng Zhang, Tun Lu, Li Shang, and Ning Gu. 2025. Enhancing LLM-Based Recommendations Through Personalized Reasoning. *arXiv preprint arXiv:2502.13845* (2025).

[33] Sijia Liu, Jiahao Liu, Hansu Gu, Dongsheng Li, Tun Lu, Peng Zhang, and Ning Gu. 2023. Autoseqrec: Autoencoder for efficient sequential recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 1493–1502.

[34] Bamshad Mobasher, Robin Burke, Runa Bhaumik, and Jeff J Sandvig. 2007. Attacks and remedies in collaborative recommendation. *IEEE Intelligent Systems* 22, 3 (2007), 56–63.

[35] Michael P O'Mahony, Neil J Hurley, and Guénolé CM Silvestre. 2005. Recommender systems: Attack types and strategies. In *AAAI*. 334–339.

[36] Alec Radford, Luke Metz, and Soumith Chintala. 2015. Unsupervised representation learning with deep convolutional generative adversarial networks.

[37] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018).

[38] Francesco Ricci, Lior Rokach, and Bracha Shapira. 2010. Introduction to recommender systems handbook. In *Recommender systems handbook*. Springer, 1–35.

[39] Carlos E Seminario and David C Wilson. 2016. Nuking Item-Based Collaborative Recommenders with Power Items and Multiple Targets.. In *FLAIRS*. 560–565.

[40] Wentao Shi, Xiangnan He, Yang Zhang, Chongming Gao, Xinyue Li, Jizhi Zhang, Qifan Wang, and Fuli Feng. 2024. Large language models are learnable planners for long-term recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1893–1903.

[41] Anu Shrestha, Francesca Spezzano, and Maria Soledad Pera. 2021. *An empirical analysis of collaborative recommender systems robustness to shilling attacks*.

[42] Yubo Shu, Haonan Zhang, Hansu Gu, Peng Zhang, Tun Lu, Dongsheng Li, and Ning Gu. 2024. RAH! RecSys–Assistant–Human: A Human-Centered Recommendation Framework With LLM Agents. *IEEE Transactions on Computational Social Systems* (2024).

[43] Junshuai Song, Zhao Li, Zehong Hu, Yucheng Wu, Zhenpeng Li, Jian Li, and Jun Gao. 2020. Poisonrec: an adaptive data poisoning framework for attacking black-box recommender systems. In *2020 IEEE 36th international conference on data engineering (ICDE)*. IEEE, 157–168.

[44] Agnideven Palanisamy Sundar, Feng Li, Xukai Zou, Tianchong Gao, and Evan D Russomanno. 2020. Understanding shilling attacks and their detection traits: A comprehensive survey. *IEEE Access* 8 (2020), 171703–171715.

[45] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. 2014. Sequence to sequence learning with neural networks. *Advances in neural information processing systems* 27.

[46] Jiaxi Tang, Hongyi Wen, and Ke Wang. 2020. Revisiting adversarially learned injection attacks against recommender systems. In *Proceedings of the 14th ACM Conference on Recommender Systems*. 318–327.

[47] Lei Wang, Jingsen Zhang, Hao Yang, Zhi-Yuan Chen, Jiakai Tang, Zeyu Zhang, Xu Chen, Yankai Lin, Hao Sun, Ruihua Song, et al. 2025. User behavior simulation with large language model-based agents. *ACM Transactions on Information Systems* 43, 2 (2025), 1–37.

[48] Xiaolei Wang, Xinyu Tang, Wayne Xin Zhao, Jingyuan Wang, and Ji-Rong Wen. 2023. Rethinking the evaluation for conversational recommendation in the era of large language models. *arXiv preprint arXiv:2305.13112*.

[49] Yancheng Wang, Ziyan Jiang, Zheng Chen, Fan Yang, Yingxue Zhou, Eunah Cho, Xing Fan, Xiaojiang Huang, Yanbin Lu, and Yingzhen Yang. 2023. Recmind: Large language model powered agent for recommendation.

[50] Zhefan Wang, Yuanqing Yu, Wendi Zheng, Weizhi Ma, and Min Zhang. 2024. Macrec: A multi-agent collaboration framework for recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2024)*. 2760–2764.

[51] Chad Williams, Bamshad Mobasher, Robin Burke, Jeff Sandvig, and Runa Bhaumik. 2006. Detection of obfuscated attacks in collaborative recommender systems. In *Proceedings of the ECAI*, Vol. 6.

[52] David C Wilson and Carlos E Seminario. 2013. When power users attack: assessing impacts in collaborative recommender systems. In *Proceedings of the 7th ACM conference on Recommender systems*. 427–430.

[53] Fan Yang, Min Gao, Junliang Yu, Yuqi Song, and Xinyi Wang. 2018. Detection of shilling attack based on bayesian model and user embedding. In *2018 IEEE 30th International Conference on Tools with Artificial Intelligence (ICTAI)*. IEEE, 639–646.

[54] Shiyi Yang, Lina Yao, Chen Wang, Xiwei Xu, and Liming Zhu. 2023. Incorporated Model-Agnostic Profile Injection Attacks on Recommender Systems. In *2023 IEEE International Conference on Data Mining (ICDM)*. IEEE, 1481–1486.

[55] An Zhang, Yuxin Chen, Leheng Sheng, Xiang Wang, and Tat-Seng Chua. 2024. On generative agents in recommendation. In *Proceedings of the 47th international ACM SIGIR conference on research and development in Information Retrieval*. 1807–1817.

[56] Jizhi Zhang, Keqin Bao, Wenjie Wang, Yang Zhang, Wentao Shi, Wanhong Xu, Fuli Feng, and Tat-Seng Chua. 2024. Prospect personalized recommendation on large language model-based agent platform.

[57] Junjie Zhang, Yupeng Hou, Ruobing Xie, Wenqi Sun, Julian McAuley, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2024. Agentcf: Collaborative learning with autonomous language agents for recommender systems. In *Proceedings of the ACM Web Conference 2024*. 3679–3689.

[58] Junjie Zhang, Ruobing Xie, Yupeng Hou, Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2023. Recommendation as instruction following: A large language model empowered recommendation approach. *ACM Transactions on Information Systems* (2023).

[59] Yongfeng Zhang, Yunzhi Tan, Min Zhang, Yiqun Liu, Tat-Seng Chua, and Shaoping Ma. 2015. Catch the Black Sheep: Unified Framework for Shilling Attack Detection Based on Fraudulent Action Propagation.. In *IJCAI*, Vol. 15. 2408–2414.

[60] Zijian Zhang, Shuchang Liu, Ziru Liu, Rui Zhong, Qingpeng Cai, Xiangyu Zhao, Chunxu Zhang, Qidong Liu, and Peng Jiang. 2025. Llm-powered user simulator for recommender system. 13339–13347 pages.

[61] Yuyue Zhao, Jiancan Wu, Xiang Wang, Wei Tang, Dingxian Wang, and Maarten De Rijke. 2024. Let me do it for you: Towards llm empowered recommendation via tool learning. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2024)*. 1796–1806.

[62] Lixi Zhu, Xiaowen Huang, and Jitao Sang. 2024. How reliable is your simulator? analysis on the limitations of current llm-based user simulators for conversational recommendation. In *Companion Proceedings of the ACM Web Conference 2024*. 1726–1732.

[63] Lixi Zhu, Xiaowen Huang, and Jitao Sang. 2025. A LLM-based controllable, scalable, human-involved user simulator framework for conversational recommender systems. 4653–4661 pages.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009